



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование



gRPC



Цифровые технологии управления
транспортными процессами

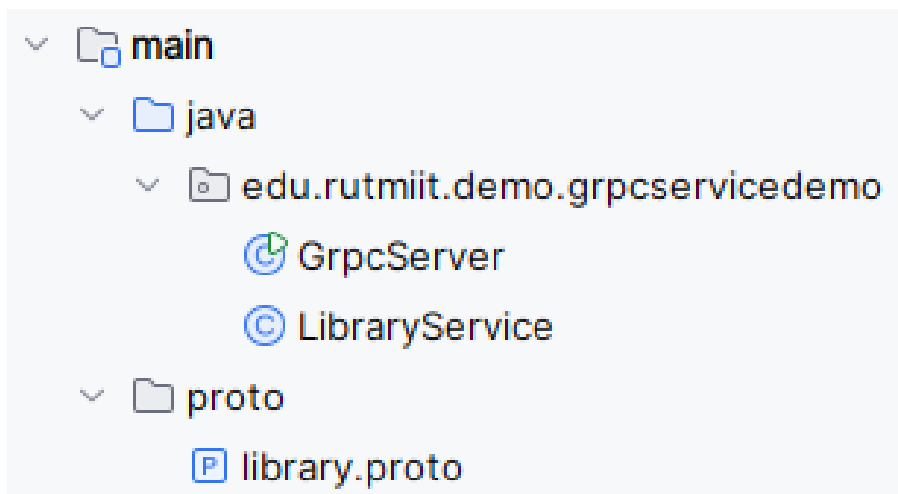
Сервис-ориентированное
программирование



Service



gRPC





src/main/proto/library.proto

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "edu.rutmiit.demo.grpcservicedemo";
option java_outer_classname = "LibraryProtos";
option optimize_for = SPEED;

import "google/protobuf/empty.proto";
package library;

service LibraryService {
  rpc AllBooks (google.protobuf.Empty) returns (GetBooksResponse) {}
  rpc GetBook (GetBookRequest) returns (GetBookResponse) {}
  rpc CreateBook (CreateBookRequest) returns (google.protobuf.Empty) {}
  rpc UpdateBook (UpdateBookRequest) returns (google.protobuf.Empty) {}
  rpc DeleteBook (DeleteBookRequest) returns (google.protobuf.Empty) {}
}
```

2024. Технологии разработки ПО.



src/main/proto/library.proto

```
message Book {  
  string isbn = 1;  
  string title = 2;  
  string author = 3;  
  string publisher = 4;  
  int32 year = 5;  
  enum BookGenre {  
    FICTION = 0;  
    NON_FICTION = 1;  
    SCIENCE = 2;  
    TECHNOLOGY = 3;  
  }  
  BookGenre genre = 6;  
}
```



src/main/proto/library.proto

```
message GetBooksResponse {  
    repeated Book books = 1;  
}  
  
message GetBookRequest {  
    string isbn = 1;  
}  
  
message GetBookResponse {  
    Book book = 1;  
}  
  
message CreateBookRequest {  
    Book book = 1;  
}  
  
message UpdateBookRequest {  
    string isbn = 1;  
    Book book = 2;  
}  
  
message DeleteBookRequest {  
    string isbn = 1;  
}
```



src/main/proto/library.proto

```
message GetBooksResponse {  
    repeated Book books = 1;  
}  
  
message GetBookRequest {  
    string isbn = 1;  
}  
  
message GetBookResponse {  
    Book book = 1;  
}  
  
message CreateBookRequest {  
    Book book = 1;  
}  
  
message UpdateBookRequest {  
    string isbn = 1;  
    Book book = 2;  
}  
  
message DeleteBookRequest {  
    string isbn = 1;  
}
```



LibraryService.java

```
public class LibraryService extends LibraryServiceGrpc.LibraryServiceImplBase {
    private static final Logger logger = LoggerFactory.getLogger(LibraryService.class);
    private final Map<String, Book> books = new HashMap<>();

    public LibraryService() {
        initializeBooks();
    }

    private void initializeBooks() {
        books.put("978-0134685991", Book.newBuilder()
            .setIsbn("978-0134685991")
            .setTitle("Effective Java")
            .setAuthor("Joshua Bloch")
            .setPublisher("Addison-Wesley")
            .setYear(2018)
            .setGenre(Book.BookGenre.TECHNOLOGY)
            .build());
    }
}
```



LibraryService.java

```
books.put("978-0596009205", Book.newBuilder()  
    .setIsbn("978-0596009205")  
    .setTitle("Head First Design Patterns")  
    .setAuthor("Eric Freeman")  
    .setPublisher("O'Reilly")  
    .setYear(2004)  
    .setGenre(Book.BookGenre.TECHNOLOGY)  
    .build());  
  
}  
@Override  
public void allBooks(Empty request, StreamObserver<GetBooksResponse> responseOb-  
server) {  
    logger.info("AllBooks gRPC API called");  
    GetBooksResponse.Builder builder = GetBooksResponse.newBuilder();  
    books.values().forEach(builder::addBooks);  
    responseObserver.onNext(builder.build());  
    responseObserver.onCompleted();  
}
```



LibraryService.java

```
@Override
public void getBook(GetBookRequest request, StreamObserver<GetBookResponse> responseOb-
server) {
    logger.info("GetBook gRPC API called");
    Book book = books.get(request.getIsbn());
    if (book == null) {
        responseObserver.onError(new StatusException(Status.NOT_FOUND));
    } else {
        responseObserver.onNext(GetBookResponse.newBuilder().setBook(book).build());
    }
    responseObserver.onCompleted();
}
```



LibraryService.java

```
@Override
public void createBook(CreateBookRequest request, StreamObserver<Empty> responseOb-
server) {
    logger.info("CreateBook gRPC API called");
    String isbn = request.getBook().getIsbn();
    if (books.containsKey(isbn)) {
        responseObserver.onError(new StatusException(Status.ALREADY_EXISTS));
    } else {
        books.put(isbn, request.getBook());
        responseObserver.onNext(Empty.newBuilder().build());
    }
    responseObserver.onCompleted();
}
```



LibraryService.java

```
@Override
public void updateBook(UpdateBookRequest request, StreamObserver<Empty> responseOb-
server) {
    logger.info("UpdateBook gRPC API called");
    if (books.containsKey(request.getIsbn())) {
        books.put(request.getIsbn(), request.getBook());
        responseObserver.onNext(Empty.newBuilder().build());
    } else {
        responseObserver.onError(new StatusException(Status.NOT_FOUND));
    }
    responseObserver.onCompleted();
}
```



LibraryService.java

```
@Override
public void deleteBook(DeleteBookRequest request, StreamObserver<Empty> responseOb-
server) {
    logger.info("DeleteBook gRPC API called");
    Book removed = books.remove(request.getIsbn());
    if (removed == null) {
        responseObserver.onError(new StatusException(Status.NOT_FOUND));
    } else {
        responseObserver.onNext(Empty.newBuilder().build());
    }
    responseObserver.onCompleted();
}
}
```



GrpcServer.java

```
public class GrpcServer {  
    public static void main(String[] args) throws Exception {  
        Server server = ServerBuilder  
            .forPort(8080)  
            .addService(new LibraryService())  
            .build();  
  
        server.start();  
        System.out.println("Server started on port 8080");  
        server.awaitTermination();  
    }  
}
```



Сгенерируйте код из proto-файла:

```
./gradlew generateProto
```



Скомпилируйте проект:

```
./gradlew build
```



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование



Client



Client

```
└─ main
  └─ java
    └─ edu.rutmiit.demo.grpcclientdemo
      └─ LibraryClient
  └─ proto
    └─ library.proto
```



LibraryClient.java

```
public class LibraryClient implements AutoCloseable {  
    private final ManagedChannel channel;  
    private final LibraryServiceGrpc.LibraryServiceBlockingStub blockingStub;  
  
    public LibraryClient(String host, int port) {  
        this.channel = ManagedChannelBuilder  
            .forAddress(host, port)  
            .usePlaintext()  
            .build();  
        this.blockingStub = LibraryServiceGrpc.newBlockingStub(channel);  
    }  
}
```



LibraryClient.java

```
@Override
public void close() throws Exception {
    try {
        channel.shutdown().awaitTermination(5, TimeUnit.SECONDS);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        System.err.println("Thread interrupted while shutting down channel");
    }
}

public static void main(String[] args) {
    try (LibraryClient client = new LibraryClient("localhost", 8080)) {
        client.demonstrateAllOperations();
    } catch (Exception e) {
        System.err.println("Error occurred: " + e.getMessage());
        e.printStackTrace();
    }
}
```



LibraryClient.java

```
public void demonstrateAllOperations() {
    try {
        System.out.println("=== Demonstrating Library Service Operations ===");
        System.out.println("\n1. Getting all books:");
        try {
            GetBooksResponse allBooks =
blockingStub.allBooks(Empty.getDefaultInstance());
            allBooks.getBooksList().forEach(book ->
                System.out.println("Found book: " + book.getTitle() + " by " +
book.getAuthor()));
        } catch (StatusRuntimeException e) {
            System.err.println("Failed to get all books: " + e.getStatus());
        }
    }
}
```



LibraryClient.java

```
System.out.println("\n2. Creating a new book:");
Book newBook = Book.newBuilder()
    .setIsbn("978-0321356680")
    .setTitle("Java Concurrency in Practice")
    .setAuthor("Brian Goetz")
    .setPublisher("Addison-Wesley")
    .setYear(2006)
    .setGenre(Book.BookGenre.TECHNOLOGY)
    .build();

try {
    blockingStub.createBook(CreateBookRequest.newBuilder().setBook(newBook).
build());

    System.out.println("Created new book: " + newBook.getTitle());
} catch (StatusRuntimeException e) {
    System.err.println("Failed to create book: " + e.getStatus());
}
```



LibraryClient.java

```
System.out.println("\n3. Getting specific book:");  
try {  
    GetBookResponse bookResponse = blockingStub.getBook(  
        GetBookRequest.newBuilder().setIsbn("978-0321356680").build());  
    System.out.println("Retrieved book: " + bookResponse.getBook().getTitle());  
} catch (StatusRuntimeException e) {  
    System.err.println("Failed to get book: " + e.getStatus());  
}
```



LibraryClient.java

```
System.out.println("\n4. Updating book:");
Book updatedBook = Book.newBuilder(newBook)
    .setYear(2007)
    .build();
try {
    blockingStub.updateBook(UpdateBookRequest.newBuilder()
        .setIsbn("978-0321356680")
        .setBook(updatedBook)
        .build());
    System.out.println("Updated book year to: " + updatedBook.getYear());
} catch (StatusRuntimeException e) {
    System.err.println("Failed to update book: " + e.getStatus());
}
```



LibraryClient.java

```
System.out.println("\n5. Deleting book:");
try {
    blockingStub.deleteBook(DeleteBookRequest.newBuilder()
        .setIsbn("978-0321356680")
        .build());
    System.out.println("Deleted book: " + updatedBook.getTitle());
} catch (StatusRuntimeException e) {
    System.err.println("Failed to delete book: " + e.getStatus());
}
// Небольшая пауза перед завершением

Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    System.err.println("Client interrupted: " + e.getMessage());
}
}
```



Сгенерируйте код из proto-файла:

```
./gradlew generateProto
```



Скомпилируйте проект:

```
./gradlew build
```



Стартуем все – сервер и клиент

```
14:00:16.637 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyClientHandler -- [id: 0x6207f0b0,
14:00:16.637 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyClientHandler -- [id: 0x6207f0b0,
Updated book year to: 2007

5. Deleting book:
14:00:16.641 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyClientHandler -- [id: 0x6207f0b0,
14:00:16.641 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyClientHandler -- [id: 0x6207f0b0,
14:00:16.644 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyClientHandler -- [id: 0x6207f0b0,
14:00:16.644 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyClientHandler -- [id: 0x6207f0b0,
14:00:16.644 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyClientHandler -- [id: 0x6207f0b0,
Deleted book: Java Concurrency in Practice
14:00:17.647 [grpc-nio-worker-ELG-1-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyClientHandler -- [id: 0x6207f0b0,
NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
raryService -- DeleteBook gRPC API called
NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
14:00:16.643 [grpc-nio-worker-ELG-3-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
14:00:17.648 [grpc-nio-worker-ELG-3-2] DEBUG io.grpc.netty.shaded.io.grpc.netty.NettyServerHandler -- [id: 0xc211b199, L:/127.0.0.1:8080 - R:
```



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование



Будут вопросы? Не стесняйтесь!

